

Dynamic Pricing Using RL

Agent-environment loop

RL has a simple repeating cycle: Agent observes STATE → Agent picks ACTION → Environment returns REWARD + new STATE → repeat

State	Last round's price
Action	The price set in this round
Reward	Profit in this round
Environment	The market computes demand, split sales, return profits
Episode	One full pricing game: could be a single round or repeated rounds

Concept 2: Linear demand

Demand curve: $Q = a - bP$

Q = quantity demanded

P = price

a = maximum quantity (price = 0)

b = elasticity of demand with respect to price

For two firms competing on price (Bertrand-style), demand for firm i depends on both prices, the firm with the lower price gets the full demand at that price level, the higher-priced firm gets little or nothing:

This is the *exact same logic* as Week 1's Bertrand profit function, just with demand made explicit instead of a generic $D(p)$. Profit is then:

$$\begin{aligned} Q_i &= a - bp_i && \text{if } p_i < p_j \\ &= \frac{1}{2}(a - bp_i) && \text{if } p_i = p_j \\ &= 0 && \text{if } p_i > p_j \end{aligned}$$

Profit is then: $\pi_i = (p_i - c) \cdot Q_i$

Here still $p_1^* = p_2^* = c$

The Gym/Gymnasium interface

Gymnasium (the maintained fork of OpenAI Gym) standardizes how environments work so any RL algorithm can plug into any environment. Every environment needs:

- `__init__`: define `observation_space` and `action_space`
- `reset()`: start a new episode, return initial observation
- `step(action)`: take an action, return (observation, reward, terminated, truncated, info)
- `render()`: visualize what's happening

`__init__` : configuration and spaces

- Pass `a,b,c` as parameters with defaults, so we can re-run experiments later without editing the class itself.
- Gym action spaces work best as either Discrete (a fixed menu of choices) or Box (continuous). Continuous pricing is realistic but it makes Q-learning harder, since tabular Q-learning needs discrete states/actions. So discretizing price into a grid 25–50 evenly spaced price levels between 0 and some max price.

Observation space: what an agent gets to see each round. Minimal version: last round's two prices, normalized to `[0, 1]`. That's a Box space of shape `(2,)`.

Action space: each firm picks one of `n_price_levels` discrete prices. That's `Discrete(n_price_levels)` *per agent*. Since Gymnasium's base API is single-agent, for two firms we have two clean options:

1. Make `step()` accept a tuple/array of two actions (multi-agent via one combined env)
2. Use a proper multi-agent wrapper (PettingZoo)

`reset()`

Called at the start of every episode. Needs to:

- Reset step counter to 0
- Pick or reset initial prices (often just zeros, or random)
- Return the first observation

`step(actions)` — the core logic

- Converting each agent's discrete action index → actual price (e.g., action 12 out of 25 levels → price $12 \times (\text{max_price}/24)$)
- Compute quantity for each firm using the piecewise demand rule (lower price wins, ties split)
- Compute profit: $(p_i - c) \times Q_i$
- Reward = profit (this round)
- New observation = the new prices
- `terminated` = False (no natural end), `truncated` = True once `max_steps` reached

The unit test

Before any agent exists: manually calling `step()` with both firms set to action = the index corresponding to price = marginal cost. Profit should come out to ~ 0 for both. Then one firm slightly above cost, one at cost — the at-cost firm should capture full demand and the other should get ~ 0 demand. If those two checks pass, your environment encodes Bertrand correctly. If they don't, nothing downstream will be trustworthy.